

06Bike-Sharing-EDA

October 5, 2021

1 Data Science: Bridging Principles and Practice

1.1 Part 6: Preparing to Model (Bike Sharing case study)

```
<div style="float:left;width:40%">
  
</div>
<div style="float:left;width:40%">
  
</div>
```

In this notebook, we will introduce the Capital Bike Sharing prediction problem and go through two major steps that occur before we build a machine learning model: Exploratory Data Analysis (EDA) and the test-train split.

1.1.1 Table of Contents

```
<li><a href="#section0">Preparing to Model</a>
  <ol type=a>
    <br>
    <li><a href="#sectioncase">Case Study: Bike Sharing</a></li>
    <br>
    <li><a href="#section1">Exploratory Data Analysis</a>
      <ul>
        <li><a href="#subsection1a">Data Types and Summary Statistics</a></li>
        <br>
        <li><a href="#subsection1b">Exploratory Visualizations</a></li>
      </ul>
    </li>
    <br>
    <li><a href="#section2">The Test-Train Split</a></li>
  </ol>
</li>
</ol>
```

```
In [1]: # run this cell to import some necessary packages
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
%matplotlib inline
import seaborn as sns
from sklearn.model_selection import train_test_split

import ipywidgets as widgets
sns.set(rc={'figure.figsize':(11.7,8.27)})
import warnings
warnings.filterwarnings('ignore')

```

1.2 6a. Case Study: Capital Bike Share

Bike-sharing systems have become increasingly popular worldwide as environmentally-friendly solutions to traffic congestion, inadequate public transit, and the "last-mile" problem. Capital Bikeshare runs one such system in the Washington, D.C. metropolitan area.

The Capital Bikeshare system comprises docks of bikes, strategically placed across the area, that can be unlocked by *registered* users who have signed up for a monthly or yearly plan or by *casual* users who pay by the hour or day. They collect data on the number of casual and registered users per hour and per day.

Let's say that Capital Bikeshare is interested in a **prediction** problem: predicting how many riders they can expect to have on a given day. [UC Irvine's Machine Learning Repository](#) has combined the bike sharing data with information about weather conditions and holidays to try to answer this question.

In this notebook, we'll walk through the steps a data scientist would take to answer this question.

1.3 6b. Exploratory Data Analysis (EDA)

"It is important to understand what you CAN DO before you learn to measure how WELL you seem to have done it." -John W. Tukey, *Exploratory Data Analysis*

Exploratory Data Analysis (EDA) is the process of 'looking at data to see what it seems to say'. EDA is an essential first step toward answering any research question. Through this process, we hope to accomplish several things: - learn about the overall 'shape' of the data: structure, organization, ranges of values - assess what assumptions we can make about the data as a basis for later statistical inference - figure out the appropriate tools and techniques for analysis - tentatively create testable, appropriate hypotheses or models

We will do this by looking at summary statistics and visualizations of the different variables.

1.3.1 Data Types

Before we know *how* to visualize the data, we need to know what types of data we're working with. Run the following cell to show our bike sharing training data.

```

In [2]: # run this cell to load the data
        # load the data from a csv file
        bikes = pd.read_csv("data/day_renamed_dso.csv", index_col=0)

```

```
# convert the dates to Python datetime objects for easier sorting
bikes["date"] = pd.to_datetime(bikes["date"])

# show the first 5 rows
bikes.head()
```

```
Out[2]:
```

	date	season	year	month	is holiday	week day	is work day	\
instant								
1	2011-01-01	spring	2011	1	no	saturday	no	
2	2011-01-02	spring	2011	1	no	sunday	no	
3	2011-01-03	spring	2011	1	no	monday	yes	
4	2011-01-04	spring	2011	1	no	tuesday	yes	
5	2011-01-05	spring	2011	1	no	wednesday	yes	

	weather	temp	felt temp	humidity	windspeed	casual riders	\
instant							
1	2	0.344167	0.363625	0.805833	0.160446	331	
2	2	0.363478	0.353739	0.696087	0.248539	131	
3	1	0.196364	0.189405	0.437273	0.248309	120	
4	1	0.200000	0.212122	0.590435	0.160296	108	
5	1	0.226957	0.229270	0.436957	0.186900	82	

	registered riders	total riders
instant		
1	654	985
2	670	801
3	1229	1349
4	1454	1562
5	1518	1600

A few of the less straight-forward columns can be described as follows: - **instant**: record index - **is holiday** : "yes" if day is a holiday, "no" otherwise - **is work day** : "yes" if day is not a weekend or holiday, otherwise "no" - **weather** : - 1: Clear, Few clouds, Partly cloudy, Partly cloudy - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds - 4: Heavy Rain + Ice Pellets + Thunderstorm + Mist, Snow + Fog - **temp** : Normalized temperature in Celsius. The values are derived via $(t-t_{min})/(t_{max}-t_{min})$, $t_{min}=-8$, $t_{max}=+39$ (only in hourly scale) - **felt temp**: Normalized feeling temperature in Celsius. The values are derived via $(t-t_{min})/(t_{max}-t_{min})$, $t_{min}=-16$, $t_{max}=+50$ (only in hourly scale) - **humidity**: Normalized humidity. The values are divided to 100 (max) - **windspeed**: Normalized wind speed. The values are divided to 67 (max) - **casual**: count of casual users - **registered**: count of registered users - **total riders**: count of total rental bikes (casual + registered)

QUESTION: Which of the variables are numerical and which are categorical? Intuitively, which do you think would be useful for predicting the number of riders on a given day? Would you choose different variables depending on if you wanted to predict casual versus registered rider counts?

ANSWER: Numerical - temp, felt temp, humidity, windspeed, casual riders, registered riders, total riders Categorical - is holiday, is work day, weather, season, year, month

1.3.2 Summary Statistics

It can also be useful to know some *summary statistics* about the different variables: things like the minimum, maximum, and average. Earlier, we learned how to do this on individual columns using functions like `np.min`, `np.max`, and `np.average`.

Thankfully, we can generate a variety of summary statistics for many variables at the same time using a `DataFrame` method called `describe`. Run the following cell to generate the summary statistics using `describe`.

For each variable with numerical values, `describe` will give: - the *count* of non-missing values - the *mean* (average) of the variable - the *standard deviation* - the 25th, 50th, and 75th *percentiles* - the *minimum* and *maximum* values

```
In [3]: # generate summary statistics
        bikes.describe()
```

```
Out [3]:
```

	year	month	weather	temp	felt temp	\
count	731.000000	731.000000	731.000000	731.000000	731.000000	
mean	2011.500684	6.519836	1.395349	0.495385	0.474354	
std	0.500342	3.451913	0.544894	0.183051	0.162961	
min	2011.000000	1.000000	1.000000	0.059130	0.079070	
25%	2011.000000	4.000000	1.000000	0.337083	0.337842	
50%	2012.000000	7.000000	1.000000	0.498333	0.486733	
75%	2012.000000	10.000000	2.000000	0.655417	0.608602	
max	2012.000000	12.000000	3.000000	0.861667	0.840896	

	humidity	windspeed	casual riders	registered riders	total riders
count	731.000000	731.000000	731.000000	731.000000	731.000000
mean	0.627894	0.190486	848.176471	3656.172367	4504.348837
std	0.142429	0.077498	686.622488	1560.256377	1937.211452
min	0.000000	0.022392	2.000000	20.000000	22.000000
25%	0.520000	0.134950	315.500000	2497.000000	3152.000000
50%	0.626667	0.180975	713.000000	3662.000000	4548.000000
75%	0.730209	0.233214	1096.000000	4776.500000	5956.000000
max	0.972500	0.507463	3410.000000	6946.000000	8714.000000

QUESTION: Looking at these statistics as data scientists, we're interested in a few things in particular:

- are there any values missing (e.g. days for which some data was not collected)? Look at the
 - what ranges of values does each variable take?
 - are there any extreme values that might throw off our analysis?
-

Using the summary table, answer these questions below.

ANSWER: Number of riders have a big range and may have some outliers

1.3.3 Visualization Continued: Numerical Data and Widgets

So far, we've worked largely with categorical variables, which we visualized with bar graphs. The bike sharing data contains several *numerical* variables, which will necessitate different visualizations.

You've previously used the `hist` function to visualize the distribution of a numerical variable. The following cell creates a **widget** that will make different histograms based on the variable you choose in the drop box. Run the cell to create the widget (don't worry too much about the details of the code).

```
In [4]: # make the plots bigger
sns.set(rc={'figure.figsize':(11.7,8.27)})

# create a widget to plot and compare different histograms
explanatory_slider = widgets Dropdown(options=["temp", "felt temp", "humidity", "windspeed"],
display(widgets.interactive(lambda x: bikes.hist(x, bins=30), x=explanatory_slider))

interactive(children=(Dropdown(description='x', options=('temp', 'felt temp', 'humidity', 'windspeed')),
```

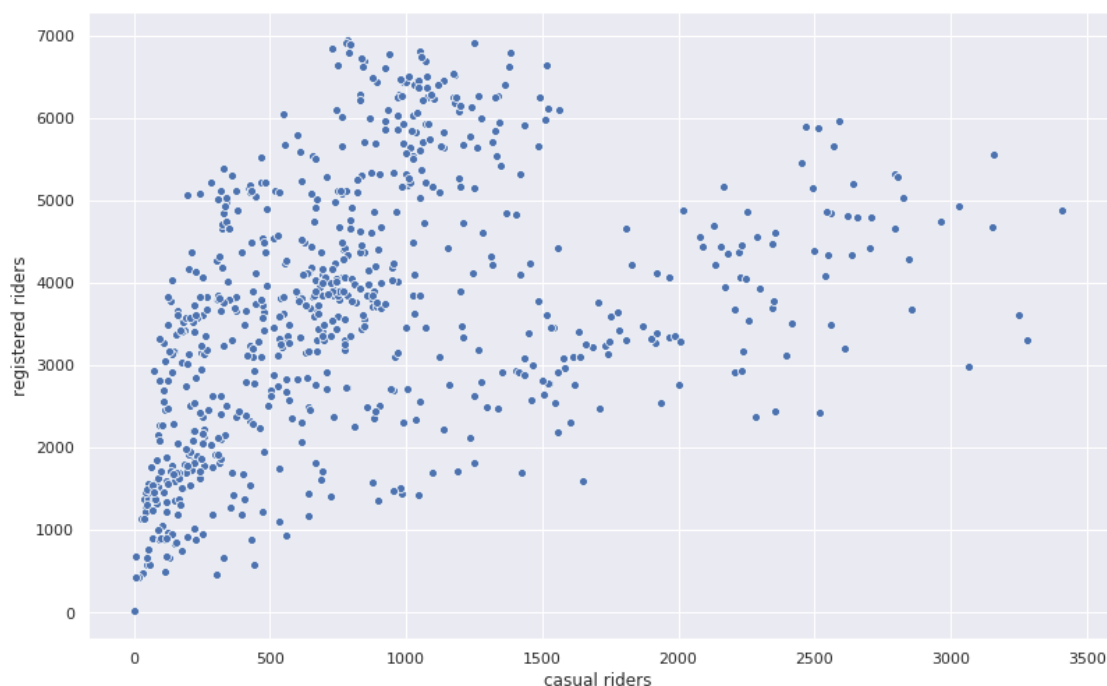
QUESTION: Describe the distributions of the different variables. Are they normally distributed? Are any of them skewed (that is, do any of them have most of their values to the left or right of the histogram's center)? What values do each of them take on?

ANSWER: Temp and Felt temp look bimodal

To predict the number of riders (the **response variable**) based on an **explanatory variable**, we often want to plot them against one another. This is often best done using a scatter plot.

`scatterplot` is a Seaborn function that creates a scatter plot of one numerical variable versus another. The first argument, `x`, is the name of the variable to be plotted on the x-axis. The second, `y`, is the name of the variable on the y-axis. Finally, we assign our DataFrame of bikes data to the `data` argument.

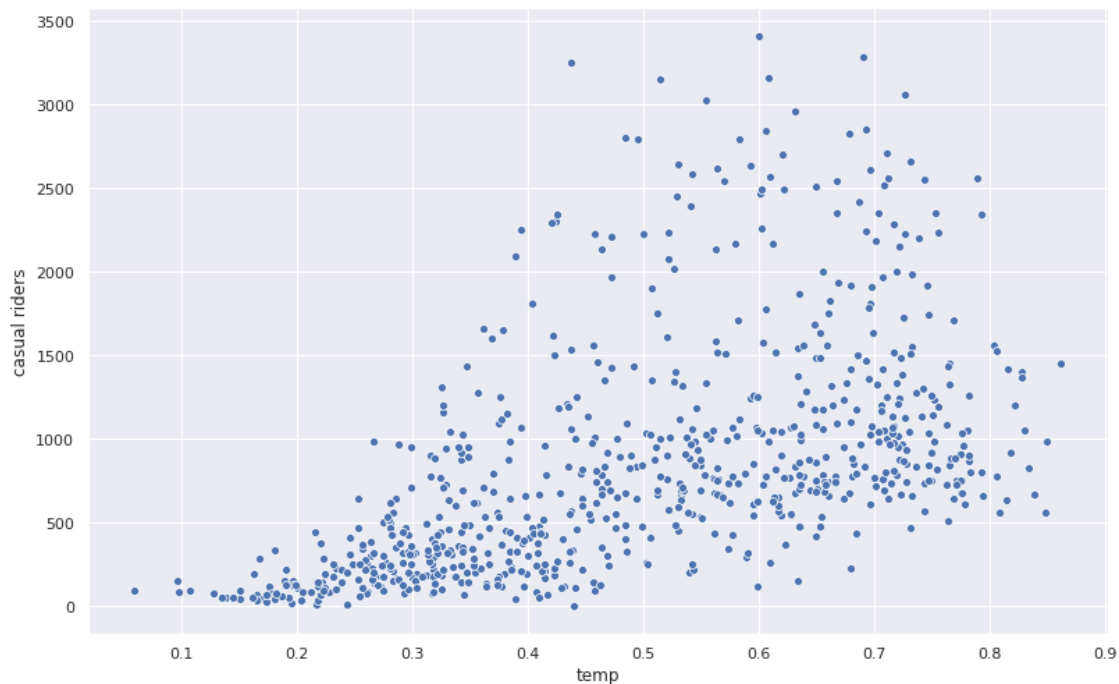
```
In [5]: # example of scatter: plot the number of casual riders against registered riders
sns.scatterplot(x="casual riders", y="registered riders",
data=bikes);
```



The shape of the scatter plot can give us some information about how two variables are **correlated**: what the value of one variable can or cannot tell you about the value of another.

EXERCISE: Try plotting at least one numerical explanatory variable (temp, felt temp, wind-speed, or humidity) against a response variable (casual, registered, or total riders). What would you say about the relationship between the two variables based on the scatter plot?

```
In [6]: # Fill in the ellipses with your code
sns.scatterplot(x="temp", y="casual riders",
               data=bikes);
```



You can also look for patterns by grouping variables. For instance, we might want to look at behavior for casual riders on work days and compare that to non-work days. We can do this by adding the "is work day" variable as the *hue* of the scattered points to get a different color for each type of day.

```
In [7]: # plot temperature against casual riders, grouped by work day type
sns.scatterplot(x="temp", y="casual riders",
               data=bikes, hue="is work day");
```



In the following cell, we've created another widget to make it easier to compare multiple variables against one another.

```
In [8]: # create a widget to make different scatter plots
        explanatory_slider = widgets Dropdown(options=["date", "temp", "felt temp", "humidity"])
        def scatter_widget(var):
            sns.scatterplot(x=var, y="registered riders", data=bikes,
                           label="registered riders")
            sns.scatterplot(x=var, y="casual riders", data=bikes,
                           label="casual riders")
            plt.xlim(bikes[var].min(), bikes[var].max())
            plt.legend()

        widgets.interact(scatter_widget, var=explanatory_slider);

interactive(children=(Dropdown(description='var', options=('date', 'temp', 'felt temp', 'humidity'))
```

QUESTION: Based on the scatter plots, which variables appear to be linearly correlated with rider counts? Which variables appear to be non-linearly correlated or uncorrelated? Is the apparent correlation different for casual or registered riders?

ANSWER: Temperature has linear correlation with riders

Finally, we want to visualize our categorical variables using bar graphs. Remember, for categorical variables we are grouping rows into the different possible categories (like the seven days of the week) and aggregating all the values in the group into a single value (in this case, taking the average).

Run the next cell to create a widget for making the different bar plots, with different color bars for each type of rider.

```
In [9]: # define a function that groups by an explanatory variable and creates a bar graph
def bar_bikes(expl_var):
    bikes_categ = pd.read_csv("data/bikes_categorical.csv")
    sns.barplot(x=expl_var, y="riders", data=bikes_categ,
                hue="rider type")

    explanatory_menu = widgets Dropdown(options=[ "season", "month", "is holiday", "week d
                                                "weather"])
    display(widgets.interactive(bar_bikes, expl_var=explanatory_menu))

interactive(children=(Dropdown(description='expl_var', options=('season', 'month', 'is holiday
```

QUESTION: Many of our categorical variables are related to time (e.g. week day, month, etc). How do usage patterns over time differ for registered and casual users? In what categories do the different user types act similarly?

ANSWER:

1.4 6c.The Test-Train Split

When we train a model on a data set, we run the risk of **over-fitting**. Over-fitting happens when a model becomes so complex that it makes very accurate predictions for the data it was trained on, but it can't generalize to make good predictions on new data.

Over- and under-fitting are most easily explained visually. The [Scikit-Learn machine learning library](#) has a good example:

The linear model on the left is **under-fitting**: we can see that there is a lot of vertical distance (the *error*) between the actual samples (the dots) and the prediction (the blue line). The 15-degree model on the right is over-fitting: there's almost no error, but the model is so complex it is unlikely to generalize to new data. Our goal is to get the model in the middle: reduce the error as much as possible while keeping the complexity low.

We can reduce the risk of overfitting by using a **test-train split**.

1. Randomly divide our data set into two smaller sets: one for training and one for testing
2. Train the data on the training set, changing our model along the way to increase accuracy
3. Test the data's predictions using the test set.

Over-fitting to the test set By using the test set over and over to check the predictive accuracy of different models, we run the risk of over-fitting to the test set as well. In the real world, data scientists get around this by also using a **validation set**- a portion of training data that the model isn't trained on, used to find optimal *hyperparameters* for the model (parameters that are set before the model is trained). Because we're using only limited hyperparameters, and because our model is for educational purposes, we'll only use training and test sets.

The first step to dividing the data is to set the **random seed**. The random seed calibrates the pseudorandom number generator; it ensures that when we "randomly" split our data, we can replicate that "random" split in the future. This is important for **reproducibility**, which lets other scientists duplicate your analysis and assess its significance.

`np.random.seed` sets the seed. It can take any number as its argument; as long as it gets the same number in the future, it will perform "random" operations in the same way.

```
In [10]: # set the random seed
         np.random.seed(28)
```

The Scikit-Learn machine learning Python library has a function called `train_test_split` that will automatically divide the data into a test and a training set. We've done a fairly standard split here: 80% of the data is in the training set, and 20% of the data is in the test set.

```
In [11]: # run this cell to split the data into training and test DataFrames
         bike_train, bike_test = train_test_split(bikes, test_size=0.2,
                                                train_size=0.8)
```

```
In [12]: # check the shapes of the new data sets
         print("Original set shape: {}".format(bikes.shape))
         print("Training set shape: {}".format(bike_train.shape))
         print("Test set shape: {}".format(bike_test.shape))
```

Original set shape: (731, 15)

Training set shape: (584, 15)

Test set shape: (147, 15)

<p>QUESTION: Data are often expensive to collect, and having a good predictive model can be helpful. What are some reasons for putting a larger portion of data into the training set? What are some reasons for putting a smaller portion of data into the training set?</p>

<p>What are some reasons for putting a larger portion of data into the training set? What are some reasons for putting a smaller portion of data into the training set?</p>

ANSWER:

References

- Bike-Sharing data set from University of California Irvine's Machine Learning Repository <https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>
- Portions of text and code adapted from Professor Jonathan Marshall's Legal Studies 190 (Data, Prediction, and Law) course materials: [lab 2-22-18, Linear Regression](#) (Author Keeley Takimoto) and [lab 3-22-18, Exploratory Data Analysis](#) (Author Keeley Takimoto)
- "Capital Bikeshare, Washington, DC" header image by [Leeann Caferatta](#) licensed under [CC BY-ND 2.0](#)

```
In [ ]:
```